División del trabajo para 4 personas

Proyecto Aplicativo 2 – Introducción a la Inteligencia Artificial: Lógica Proposicional, Cláusulas de Horn y Motores de Inferencia Simbólica

El proyecto tiene tres frentes: la base de conocimiento (50+ reglas con restricciones lógicas fuertes), los dos motores y el informe. Esta propuesta busca que todos tengan carga parecida y puedan trabajar en paralelo desde el día uno.

Persona 1: Ingeniera/o de Conocimiento A (KB, mitad 1)

- Diseñar la **arquitectura de niveles** de la KB (síntomas → signos → síndromes → enfermedades → subtipos/alertas) y acordar con la Persona 2 la convención de nombres de los hechos (ej. `Dificultad_Respiratoria`, sin tildes ni espacios).
- Escribir las reglas de las **ramas respiratorias** (gripe, neumonía, bronquitis, COVID, etc.) con al menos 25 reglas.
- Incluir las **ramas de alta similitud**: reglas que comparten antecedentes base y se diferencian por un único hecho diferenciador.
- Fuentes: protocolos OMS / Minsalud, dataset Symptom-Disease de Kaggle.

Persona 2: Ingeniera/o de Conocimiento B (KB, mitad 2) y auditoría lógica

- Escribir las reglas de **otras ramas** (gastrointestinal, dengue/arbovirosis, neurológica, etc.), otras 25+ reglas.
- Crear las **reglas de alerta/gravedad**: diagnóstico ya inferido + síntoma crítico → estado de alerta.
- Ser responsable de la **auditoría lógica** de toda la KB con un script validador (`validar_kb.py`) que verifique:
  - mínimo 50 reglas distintas,
  - formato Horn (un solo consecuente),
  - **aciclicidad** (detección de ciclos con DFS),
  - **alcanzabilidad** (sin islas),
  - profundidad mínima de 4 niveles en los diagnósticos finales.
- Unir las dos mitades de la KB en el `base_conocimiento.json` final.

Persona 3: Motor hacia adelante y traza

- Implementar `ejecutar_inferencia()` con el algoritmo de punto fijo:
  - repetir hasta que no se agreguen hechos nuevos,
  - disparar una regla solo si todos sus antecedentes están en la memoria de trabajo y el consecuente aún no está,
  - detenerse apenas se alcance el objetivo.
- Llenar `self.traza_explicacion` con el orden cronológico de reglas disparadas (id, antecedentes, consecuente) y agregar un método para **exportar la traza** (impresión legible y/o archivo).
- Comentar el código y escribir pruebas unitarias con la KB de ejemplo (3 reglas) mientras la KB real no esté lista.

Persona 4: Motor hacia atrás, pruebas e informe

- Implementar `ejecutar_inferencia_atras()` de forma recursiva (goal-driven):
  - **caso base 1:** el objetivo está en los hechos iniciales → `True`,
  - **caso base 2:** ninguna regla tiene ese consecuente → `False`,
  - para cada regla candidata, probar todos los antecedentes como sub-objetivos,
  - **control de profundidad** y conjunto de objetivos "en proceso" para evitar bucles,
  - opcional: memoización de objetivos ya probados o fallidos.
- Liderar el **informe técnico PDF**: explicación de las estrategias algorítmicas de ambos motores.
- Diseñar y documentar los **3+ casos clínicos de prueba** (Δ y α), corriendo ambos motores y pegando las trazas paso a paso, incluyendo al menos un caso que dé `False`.

Tareas compartidas

Tarea	Responsables
Acordar convención de nombres de hechos y formato JSON	Todos (día 1)
Revisión cruzada de la KB (cada uno audita las reglas de otro)	P1 ↔ P2

Tarea	Responsables
Integración de motores con la KB final y pruebas conjuntas	P3 + P4
Revisión final del informe y del código comentado	Todos
Verificar que ambos motores den el mismo resultado en todos los casos	P3 + P4

Cronograma sugerido

1. **Día 1:** reunión conjunta. Se define la jerarquía de niveles, los nombres de hechos y quién cubre qué enfermedades. P3 y P4 arrancan con la KB de 3 reglas.
2. **Primera mitad del tiempo:** P1 y P2 escriben reglas; P3 y P4 terminan los motores. P2 entrega el validador temprano.
3. **Segunda mitad:** se unifica la KB, se valida, se corren los casos clínicos y se redacta el informe.
4. **Últimos días:** revisión cruzada, correcciones y empaquetado de entregables (`proyecto_inferencia_final.py`, `base_conocimiento.json`, informe PDF).

Consejos para evitar problemas

- **Un solo nombre por hecho.** La mayoría de los errores en este tipo de proyecto vienen de inconsistencias como `Dolor_Muscular` vs `Dolor_muscular`, que dejan reglas inalcanzables.
- Los motores deben leer `Regla.antecedentes` como `set`, así que cuidado con antecedentes duplicados en el JSON.
- Prueben el motor con casos que **no** deben concluir el objetivo; es lo que más revela errores.
- Si usan IA para generar reglas, la auditoría con el validador de P2 es obligatoria, porque el enunciado hace al grupo responsable de la lógica.